

```
# Display clean data
```

```
print("in clean Data after Removing outliers: ")
```

```
print(clean data)
```

Practical No:8

6.1] Data Normalization using min-max scaling to normalize numerical data into range a to 1 using min-max scaling normalization

$$x^{\text{prime}} = x * \frac{\text{min}}{\text{max} - \text{min}}$$

x max -x min

→

```
from Sklearn. Preprocessing import MinMaxScaler
```

```
import pandas as pd
```

```
Scaler-MinMaxScaler()
```

```
df = fd DataFramec
```

```
#Marks dataset
```

```
Marks': [5, 15, 25, 35, 45,55]
```

3)

```
x - min =df.min()
```

```
#Minimum
```

```
x max = df. max()
```

```
#Maximum
```

```
x=( df -x\ min)/(x\ max-x\ min)
```

```
# Manual
```

```
print (x)
```

```
df [ 'Min Max Scale'] = Scaler.fit_transform Cdf[[' Marks']])
```

```
print (df)
```

```
#scikit-learn Min Max Scaling.
```

5.2] Data Standardization using 2-Score to understand standardize numerical z-score transformation.

$$z = (x - \mu)/\sigma$$

→from Sklearn. Preprocessing import StandardScaler

import Pandas as pd

df = pd.DataFrame CE

"Marks": [40,50,60,70,80,90]

3)

#Make a copy of the DataFrame

df1 = df. copy()

Manual Method:

#calculate mean of marks.

mean = df ["Marks"]. mean()

calculate standard Deviation

Std = df [" Marks"].std()

calculate Z-score using the formula

df["Z-score"] =((df["Marks"]-mean)/std)

```
print(" Manual Method")
```

```
print (df1)
```

```
#Scikit-learn Method.
```

```
#create standard scaler object
```

```
Scaler Standard scaler ()
```

```
#Apply Z-score standardization using standard scaler
```

```
df ["SK-standardized_Marks"] = Scaler.fit-transform(df [[" Marks"]])
```

```
#
```

```
Display result using Scikit-learn
```

```
print C"In Scikit-Learn Methad: ")
```

```
print (df)
```

4.3] Multiple lines in Matplotlib : compare monthly sales of two products using multiple line plots.

→

```
Import matplotlib.pyplot as plt
```

```
#create a list of months
```

```
months = [ "Jon", "Feb", "Mar", "Apt", "May", "Jun"]
```

```
# store monthly sales of Prod A
```

```
Product A = [120, 150, 180, 160, 200, 220]
```

```
# store monthly Sales of Prod B
```

```
Product B = [100,130,160,170,190, 210]
```

```
#Plot the sales of Prod A
```

```
plt.plot (months, Product-A, Marker = "o", label = "Product A").
```

```
#Plot the sales of Prod B
```

```
plt.plot(months, Product-B, market = "o", label = "Product 8")
```

```
# Add title to the graph
```

```
plt. title C" Manthly Sales Comparison")
```

```
#Add label to the x-axis
```

```
plt. xlabel ("Month")
```

```
# Add label to the y-axis
```

```
plt. Ylabel C" Sales")
```

```
# Display legend for both Products
```

```
plt. legend()
```

```
# Display the graph
```

```
plt. Show()
```

6.4] Grouped bar chart: To compare student Performance in two subjects. using a grouped bar chart

→

```
import matplotlib.pyplot as plt.
```

```
#create student names.
```

```
Students = ["Amit", "Aahul", "Sneha", "Arya", "Priya"]
```

#Marks in Python

Python-marks = [75,85,65,90,80]

#Marks in statistics.

Statistics marks = [80,78,70,85,88]

#create Positions for students

X = range (len(students))

set width of bars.

width = 0.35

create bars of Python

plt.bar (Ci-width / 2 for i in x7.

Python marks, width = width, label="Python")

#create bars of stat

plt.bar (Cit width 12 for i in x1,

```
Py Statistics marks, width= width, label="Statistics")
```

```
#get student name as x-axis ticks.
```

```
plt.xticks Cx, students)
```

```
#get title
```

```
plt.title(" Student Performance in Two Subjects")
```

```
#get x-label
```

```
plt. xlabel (" Students")
```

```
# set Y-label
```

```
plt.ylabel C" Marks")
```

```
#
```

```
Display Legend
```

```
plt. legend()
```

```
# Display the chart
```

```
plt.show()
```

6-5] Histogram with different no. of bins: To understand how the number of bins affects histogram visualization

←

Import matplotlib.pyplot as Pit

Marks = [42,45, 48, 52, 55, 57, 60, 61, 63, 65, 67, 68, 70, 72, 74, 76, 78, 80, 82, 85,88, 90, 93, 96,3

plt.hist (Marks, bins = 8, edgecolor="black")

pit. title (" Students Marks distribution")

plt.xlabel ("Marks")

plt. ylabel C"Frequency")

plt.show()

Practical No: 9

6.] Multiple Box Plots

Title: compare student Marks in Multiple subjects using Box Plot.

Aim: To compare the distribution of marks in different Subjects

Algorithm: 1. Start the Program

2. Create a list containing Python marks.

3. Create a list containing statistics marks.

4. Create a list containing Data Science marks.

5. Pass all three datasets.

6. Assign Subject names as labels for the three box plots.

7. Add the title "Subject-wise Marks Distribution"

8. Label the Y-axis as "Marks"

9. Display the box plot

10. Stop the Program.

→

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
#1. Define the datasets
```

```
Python = [88, 75, 95, 90, 68]
```

```
Stats = [92, 80, 88, 85, 74]
```

```
= [85, 89, 91, 88, 70] DS
```

```
#2. Create the box plot using the datasets and subject labels.
```

```
Sns.boxplot Cdata = [python, stats, ds]). Set_xticklabels C
```

```
["Python", "statistics", "Data Science"]
```

```
#3. Add title and Y-axis label
```

```
plt. title C" Subjectwise Marks Distribution")
```

```
plt. ylabel ("Marks")
```

```
#4. Display the Plot
```

```
plt. Show()
```

- 6.2] Data Visualization using seaborn : Bar Plot, Histogram, Box Plot - and Scatter Plot

Aim: To create and visualize student Performance data using seaborn by generating a bar plot, histogram, box plot, and scatter Plot with Python..

Algorithm:

1. Import libraries.
2. create student data containing student Name, Study Hours, Marks and Performance (High, Medium, Low)
3. convert the data into a DataFrame.
4. Create a bar plot to display marks of each student
5. Create a histogram to show the distribution of marks.
6. create a box plot to visualize the spread and distribution of marks
7. create a scatter plot to show the relationship between Study hours and marks, Also categorize by Performance..
8. Add suitable titles to all plots and display them
9. Analyze the visualizations to understand student Performance and the relationship between study hours and marks

Solution: `import pandas as pd` `import matplotlib.pyplot as plt.`

`#Create student Performance data`

```
data = {  
  
    "Student-Name": ["Amit", "Rahul", "Sneha", "Priyo", "Neha",  
  
    "Riya"],  
  
    "Study-Hours": [2,4,5,3,6,7],  
  
    "Marks": [45,60,70,55,80,20]  
  
    "Performance": ["Low", "Medium", "Medium", "Low", "High", "High"]  
  
}  
  
#convert the data into a DataFrame  
  
df = pd. DataFrame (data)  
  
# Display the student data  
  
print(" Student Performance Data: ")  
  
print(df)  
  
# create a bar plot to display marks of each student
```

```
plt.figure Cfigsize = (7,4))
```

```
plt. bar Cdf [" student_Name"], df ["Maths"])
```

```
#Add title and labels
```

```
plt.title C" student Marks")
```

```
plt.x label C" Student Name")
```

```
pit. Ylabel C" Marks")
```

```
# Display the bar plot
```

```
plt. Show()
```

```
#Create a histogram to show the distribution of marks
```

```
plt.figure (figsize = (7,4))
```

```
plt. hist Cdf ["Marks"], bins = 5, edgecolor = "black")
```

```
plt. title C" Distribution of Marks")
```

```
pit. xlabel ("Marks")
```

```
plt. ylabel ("Frequency")
```

```
plt. show ()
```

```
# create a box plot to visualize the spread of Marks.
```

```
plt.figure (figsize = (6,4))
```

```
plt. boxplot Cdf [" Marks"])
```

```
plt. title C" Box plot of Marks")
```

```
plt.ylabel (" Marks")
```

```
plt. show()
```

```
#create a scatter Plot to show the relationship
```

```
plt.figure Cfigsize = (7,4))
```

```
plt. scatter Cdf ["Study_Hours"), df ["Marks"])
```

```
Plt. title (" study Hours Us Marks")
```

```
pit. xlabel (" study Hours")
```

```
plt. y label C" Marks")
```

```
plt.show()
```

0.3] Aim: To load an inbuilt Seaborn dataset and create different visualizations such as Bar plot, Histogram, Box plot, and scatter Plot to analyze the data

Dataset used: tips dataset = contains information about restaurant bills and tips.

Algorithms:

1. import libraries
2. Load the Seaborn inbuilt tips dataset.
3. Display the first few records using head().
4. Create a bar plot to compare average total bills for different days
5. Create a histogram to visualize the distribution of total bills by to bins.
6. create a box plot to study the distribution of total bills. across different days.
7. create a scatter plot

Solution:

```
import Seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
#Load the built-in seaborn tips dataset
```

```
tips = sns.load_dataset("tips")
```

```
# Display the first few records of the dataset print ("First few records of the Tips dataset :") print (tips.head())
```

```
# Bar Plot
```

```
#calculate the average total bill for each day.
```

```
avg_bill = tips.groupby("day")["total_bill"].mean()
```

```
plt.bar(avg_bill.index, avg_bill.values)
```

```
plt.title("Average Total Bill for Different Days")
```

```
plt.xlabel("Day")
```

```
plt.ylabel("Average Total Bill")
```

```
plt.show()
```

```
#Create a histogram with 10 bins
```

```
plt.hist(tips["total_bill"], bins = 10)
```

```
plt.title(" Distribution of Total Bills")
```

```
plt.xlabel ("Total Bill")
```

```
plt. Ylabel (" Frequency")
```

```
plt.show()
```

```
#
```

create a Box plot for tatal bills across different days

```
Sns. boxplot(x = "day", y = "total_bill", data = tips)
```

```
plt.title("Distribution of Total Bills Across Different Days")
```

```
plt. xlabel ("Day")
```

```
plt.ylabel ("Total Bill")
```

```
plt.show()
```

#create a Scatter Plot showing relationship between total bill and tip

```
plt. scatter Ctips ["total_bill"], tips Ctip])
```

```
plt. title C"Relationship between Total Bill and Tip")
```

```
plt.xlabel("Total Bill")
```

```
plt.ylabel("Tip")
```

```
plt.show()
```

6-4] creating a correlation Heatmap using Seaborn

Aim: To create a correlation heatmap using Python, Pandas, Matplotlib, and seaborn to analyze the relationships between Age, Study Hours, Attendance, Assignment score, and Marks of students..

Algorithm:

1. Import libraries
2. Create student data and store it in a DataFrame
3. Calculate the correlation matrix
4. Create a Seaborn heatmap
5. Display correlation values.
6. Add a title to the heatmap
7. Display the heatmap.

Solution:

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
import pandas as pd
```

```
#create a dictionary containing sample student data
```

```
Student data = {
```

```
'Student ID': [1,2,3,4,5],
```

```
'Name': ['Arya', 'shravani', 'Preina', 'Anjushri', 'Rohit'],
```

```
'Age': [21,26,23,22,29], 'Major': ['computer science', 'Os', 'AT', 'Math', 'stat'],
```

```
'EGPA': [3.8,3.4, 3.9, 3.2, 3.6]
```

```
    _
```

```
}
```

```
#convert the dictionary into a pandas DataFrame
```

```
df = pd.DataFrame(student_data)
```

```
# Display the dataset
```

```
print C" student Dataset: ")
```

```
print (df)
```

```
# Calculate the correlation matrix
```

```
corr_matrix = df. corr Cnumeric_only = True)
```

```
# Display the correlation matrix text output
```

```
print C"In --- Correlation Matrix ---")
```

```
print C cor-matrix)
```

```
# Generate the heatmap
```

```
plt. figure (figsize = (6,4))
```

```
sns. heatmap C
```

```
Corr-matrix,
```

```
annat = True,
```

#Display correlation coeff num inside cells.

Cmap = "Coolwarm", # color gradient from blue (-ve) to red (+ve)

vmin=-1, # Minimum boundary for correlation value.

Vmax = 1, # Maximum boundary for correlation value.

linewidths = 0.5, #Puts thin spaces lines between Squares.

Square = True #Keeps cells shaped like Perfect Squares

plt.title("Student Data Correlation Heatmap", fontsize= 12, fontweight =

plt.tight layout()

plt. show()

bold")

6.5] Exploratory Data Analysis using Seaborn Pair Plot

Aim: To create a Pair plot using seaborn to visualize relationships among. multiple numerical variables and analyze Student Performance based on Study hours, attendance, and marks

Algorithm:

1. Import libraries

2. Create a dataset containing Study Hours, Attendance, Marks, and Performance.
3. convert the dataset.
4. use Sns. pairplot() to create pairalise relationships between, numerical variables
5. categories by Performance category
6. Display the pair plat
7. Observe the relationships and Patterns between Study Hours, Attendance, and Marks for different Performance levels.

Solution:

Import Pandas as pd.

import Seaborn as sns

import matplotlib.pyplot as plt.

#create a Dataset

data = 2

"Study-Hours": [2345678,97,

```
"Attendance": [60,65,70,75,80,85,90,95],
```

```
"Marks": [45, 60, 58, 65, 72,80,88,95],
```

```
"
```

```
Performance": ["Low", "Low", "Medium", "Medium", "Medium"]
```

```
"High", "High", "High"]
```

```
df = pd. DataFrame(data)
```

```
print C" Student Performance Data:")
```

```
print(df)
```

```
#create a Pair plot for numerical variables.
```

```
#'hue' categorize the data according to Performance
```

```
sns. pairplot C df, vars = ["study-Hours", "Attendance", "Marks"], hue = "Performance")
```

```
#Add a title to the Pair Plot
```

```
plt. Subtitle ("Pairwise Relationship of Student Performance")
```